TP sur les types de données et la construction de data frame

Durel TEUMO

2026-04-14

Contents

1 Typage implicite, coercicion et ambigüités **1**
1.1 Question 1 – Type et mode des objets . . . 1
1.2 Question 2 – Analyse des résultats . . . 2
1.3 Question 3 – Cas de coercition implicite . . . 2
1.4 Question 4 – Type cible et logique de conversion mobilisée . . . 2
1.5 Question 5 – Discussion sur les conséquences . . . 2

2 Structures hétérogènes et propagation des types **4**
2.1 Question 1 – Déterminons le type de chaque colonne de D . . . 4
2.2 Question 2 – Explication de la validation de la stucture du data frame . . . 4
2.3 Question 3 – Comparaison du comportement de `D` . . . 4
2.4 Question 4 — Type final de la matrice . . . 4
2.5 Question 5 — Perte d'information statistique . . . 5
2.6 Question 6 — Stratégie pour préserver les types . . . 5

3 Facteurs, encodage et biais d'interprétation **6**
3.1 Question 1 – Transformation en facteur ordonné . . . 6
3.2 Question 2 – Valeurs numériques internes . . . 6
3.3 Question 3 – Danger de as.numeric(niveau) . . . 6
3.4 Question 4 – Représentation numérique cohérente . . . 6
3.5 Question 5 – Validité statistique . . . 6
3.6 Question 6 – Biais potentiel si les niveaux sont mal ordonnés . . . 7

4 Construction structurée d'un data frame **8**
4.1 Question 1 – Construction du data frame . . . 8
4.2 Question 2 – Création de la variable moyenne_annuelle . . . 8
4.3 Question 3 – Création de la variable sur le statut d'admission . . . 9
4.4 Question 4 – Pertinence de la transformation en facteurs ordonnés . . . 9
4.5 Question 6 – Contrôle automatique . . . 10
4.6 Question 7 – Implications pour les analyses statistiques . . . 10

1 Typage implicite, coercicion et ambigüités

1.1 Question 1 – Type et mode des objets

Considérons les objets définis comme suit:

```
x <- c(1,2,3)
y <- c(TRUE,FALSE,TRUE)
z <- c("4", "5", "6")
w <- list(1, "2", TRUE)
```

Ici , `x` est un **vecteur d'entiers**; `y` est un **vecteur logique**; `z` est un **vecteur de caractères** et `w` est une **liste**.

1.2 Question 2 – Analyse des résultats

Dans cette question, nous allons analyser les resultats des opérations suivantes:

```
x+y
```

```
## [1] 2 2 4
```

Ici, R a converti le vecteur logique en **combinaison numérique binaire**: 0 pour `FALSE` et 1 pour `TRUE`. Le resultat obtenu est un **vecteur d'entiers**.

```
 x+z
```

```
## Error in `x + z`:
## ! argument non numérique pour un opérateur binaire
```

On voit qu'une erreur est signalée: En effet, à cause des guillemets, R considère les valeurs de `z` comme des caractères, et de ce fait ne peut pas réaliser une addition.

```
as.numeric(z)+x
```

```
## [1] 5 7 9
```

Le code s'exécute correctement: En effet, grâce à la fonction `as.numeric()` R effectue d'abord une conversion des éléments de `z` en nombres, puis effectue l'addition.

```
unlist(w)+x
```

```
## Error in `unlist(w) + x`:
## ! argument non numérique pour un opérateur binaire
```

Le code donne une erreur. En effet, lorsqu'on applique la fonction`unlist()` à `w`, du fait de la *hiérarchie de types*, `w` va être converti en une **liste de caractères**. De ce fait, R ne peut réaliser l'addition entre des caractères et des nombres.

1.3 Question 3 – Cas de coercition implicite

Les cas de coercition implicite sont les cas de transformation automatique d'un type de donnée en un autre sans qu'on ne le demande directement. Par exemple, lorsqu'on fait `unlist(w)`, on demande sans le vouloir à R de convertir tous les éléments de `w` en caractères.

1.4 Question 4 – Type cible et logique de conversion mobilisée

Dans le premier cas, le type cible est un **vecteur numérique**. Dans le 3e, le type cible est aussi un **vecteur numérique**. Dans les 2e et 4e cas, on a une erreur.

1.5 Question 5 – Discussion sur les conséquences

Considérons l'instruction suivante:

```
as.numeric(c("10","A","20"))
```

```
## [1] 10 NA 20
```

Interprétation du résultat: Dans cette instruction, la fonction `as.numeric()` permet de, dans la mesure du possible, convertir un élément en nombre: c'est une **coercition explicite** vers le type numérique. Dans notre vecteur ci-dessus, tous les éléments sont considérés comme des caractères. Cependant, il y a une différence entre eux: les éléments "10" et "20" sont des nombres passés en caractère, donc peuvent être reconvertis en

nombres grâce à `as.numeric`. Par contre, “A” est un caractère de nature, et ne correspond à aucun nombre. D’ou le **Not Avalaible** (`NA`) pour indiquer qu’aucune conversion de “A” en entier n’est possible.

2 Structures hétérogènes et propagation des types

Considérons la construction :

```
A <- c(1,2,3)
B <- c("a", "b", "c")
C <- c(TRUE, FALSE, TRUE)
D <- data.frame(A,B,C)
```

2.1 Question 1 – Déterminons le type de chaque colonne de D

Pour ce faire, nous pouvons utiliser plusieurs fonctions sur R, comme `str` ou encore `sapply` et `class` simultanément:

```
#Méthode 1
str(D)
```

```
## 'data.frame':    3 obs. of  3 variables:
##  $ A: num  1 2 3
##  $ B: chr  "a" "b" "c"
##  $ C: logi  TRUE FALSE TRUE
```

```
#Méthode 2
sapply(D, class)
```

```
##           A           B           C
##   "numeric" "character"   "logical"
```

2.2 Question 2 – Explication de la validation de la stucture du data frame

R accepte cette tructure car D tel que défini est un `data frame`. En effet, un data frame est une liste déléments de même nature sur chaque colonne, ce qui est respecté ici, car chacun des vecteurs A, B et C est passé en colonne dans D.

2.3 Question 3 – Comparaison du comportement de `D`

Soit l'instruction:

```
M <- as.matrix(D)
```

La différence dans ce cas est que, une **matrice** est une collection d'éléments de même nature. Ainsi,du fait de la hiérarchie des types, cette instruction entraîne une coercition implicite, qui convertit tous les éléments de D en caractères.

2.4 Question 4 — Type final de la matrice

```
class(M)
```

```
## [1] "matrix" "array"
```

```
typeof(M)
```

```
## [1] "character"
```

La matrice est de type `character`. En R, une matrice doit avoir un type **unique et homogène** pour tous ses éléments. Lors de la conversion, R applique la règle de **coercition implicite** selon la hiérarchie :

$$\text{logical} < \text{integer} < \text{numeric} < \text{character}$$

Le type `character` étant le plus général, tous les éléments sont convertis vers lui.

2.5 Question 5 — Perte d'information statistique

```r
# Avant conversion : on peut faire des calculs sur la colonne A
mean(D$A)
```

```
## [1] 2
```

```r
sum(D$C)   # TRUE = 1, FALSE = 0
```

```
## [1] 2
```

```r
# Après conversion : tout est character, les calculs sont impossibles
mean(M[, "A"])   # renvoie NA avec un warning
```

```
## [1] NA
```

La conversion entraîne deux pertes majeures :

- La colonne **A** (numérique) ne permet plus aucun calcul statistique (moyenne, somme, variance…)
- La colonne **C** (logique) perd sa valeur booléenne — `"TRUE"` est désormais une simple chaîne de caractères, non interprétable comme condition logique

Dans un contexte d'analyse de données, cela peut fausser silencieusement des résultats si on ne vérifie pas les types après transformation.

2.6 Question 6 — Stratégie pour préserver les types

Pour préserver les types tout en ayant la possibilité de faire des calculs, on peut essayer de **ne travailler qu'avec les colonnes numériques:**

```r
# Extraire uniquement les colonnes numériques
cols_num <- sapply(D, is.numeric)
M_num <- as.matrix(D[, cols_num])
M_num
```

```
##      [,1]
## [1,]    1
## [2,]    2
## [3,]    3
```

```r
class(M_num)   # numeric - les calculs sont possibles
```

```
## [1] "matrix" "array"
```

La stratégie à privilégier dépend du contexte : si toutes les variables sont numériques, `as.matrix()` est sans risque. Sinon, il vaut mieux travailler colonne par colonne ou encoder explicitement les variables qualitatives.

3 Facteurs, encodage et biais d'interprétation

Considérons le vecteur :

```
niveau <- c("faible", "moyen", "élevé", "moyen", "faible")
```

3.1 Question 1 – Transformation en facteur ordonné

```
niveau_f <- factor(niveau,
                   levels = c("faible", "moyen", "élevé"),
                   ordered = TRUE)
niveau_f
```

```
## [1] faible moyen  élevé  moyen  faible
## Levels: faible < moyen < élevé
```

L'argument `levels` permet de définir la hiérarchie des modalités, et `ordered = TRUE` indique à R que ce facteur est **ordonné**. On obtient bien la relation faible < moyen < élevé.

3.2 Question 2 – Valeurs numériques internes

```
as.numeric(niveau_f)
```

```
## [1] 1 2 3 2 1
```

```
levels(niveau_f)
```

```
## [1] "faible" "moyen"  "élevé"
```

R encode les modalités en entiers selon leur rang dans `levels` : `faible` → 1, `moyen` → 2, `élevé` → 3.

3.3 Question 3 – Danger de as.numeric(niveau)

```
as.numeric(niveau)
```

```
## Warning: NAs introduits lors de la conversion automatique
```

```
## [1] NA NA NA NA NA
```

Cette instruction est dangereuse car :

`niveau` est encore un **vecteur de caractères** à ce stade. R ne peut pas le convertir en nombres, ce qui produit des `NA` pour chaque élément

3.4 Question 4 – Représentation numérique cohérente

La méthode correcte consiste à convertir le **facteur ordonné** `niveau_f` et non le vecteur de caractères brut :

```
codes_corrects <- as.numeric(niveau_f)
codes_corrects
```

```
## [1] 1 2 3 2 1
```

3.5 Question 5 – Validité statistique

```
revenu <- c(100, 200, 300, 250, 150)
```

```r
# Filtrage et calcul
mean(revenu[niveau_f == "élevé"])
```

```
## [1] 300
```

L'expression est **statistiquement valide** dans ce contexte, sous deux conditions respectées ici :

- L'opérateur `==` sur un facteur ordonné fonctionne correctement car R compare les étiquettes et non les codes internes
- `niveau_f` et `revenu` ont la **même longueur** et les observations sont **appariées** — le i-ème élément de `revenu` correspond bien au i-ème élément de `niveau_f`

```r
data.frame(niveau_f, revenu)
```

```
##   niveau_f revenu
## 1   faible    100
## 2    moyen    200
## 3    élevé    300
## 4    moyen    250
## 5   faible    150
```

On peut vérifier que seule l'observation à l'indice 3 a le niveau `élevé`, avec un revenu de 300. La moyenne est donc 300.

3.6 Question 6 – Biais potentiel si les niveaux sont mal ordonnés

```r
# Facteur avec un ordre incorrect
niveau_mal_ordonne <- factor(niveau,
                             levels = c("élevé", "faible", "moyen"),
                             ordered = TRUE)

# Les codes internes sont maintenant faux
as.numeric(niveau_mal_ordonne)
```

```
## [1] 2 3 1 3 2
```

```r
# Comparaisons ordonnées deviennent fausses
niveau_mal_ordonne > "faible"
```

```
## [1] FALSE  TRUE FALSE  TRUE FALSE
```

```r
niveau_f          > "faible"   # version correcte
```

```
## [1] FALSE  TRUE  TRUE  TRUE FALSE
```

Un mauvais ordonnancement des niveaux entraîne deux types de biais :

- Les **comparaisons ordinales** (`>`, `<`) donnent des résultats faux — par exemple `faible > moyen` pourrait être `TRUE` selon l'ordre mal défini
- Les **codes numériques internes** ne reflètent plus la hiérarchie réelle, ce qui fausse toute analyse qui s'appuie dessus (régression ordinale, visualisation par rang, tri…)

La bonne pratique est donc de **toujours définir explicitement** l'argument `levels` dans l'ordre croissant logique, et de vérifier avec `levels()` avant toute analyse.

4 Construction structurée d'un data frame

4.1 Question 1 – Construction du data frame

```r
set.seed(123)
n <- 100  # taille de la cohorte

etudiants <- data.frame(
  # Identifiant unique - caractère
  id = paste0("ETU", sprintf("%03d", 1:n)),

  # Sexe - facteur non ordonné
  sexe = factor(
    sample(c("M", "F"), n, replace = TRUE) #replace = T car c'est un tirage avec remise
  ),

  # Filière - facteur non ordonné (justification à la Q4)
  filiere = factor(
    sample(c("ISEP", "ISE", "AS"), n, replace = TRUE)
  ),

  # Moyennes semestrielles - numérique
  moy_S1   = ifelse(
              sample(c(TRUE, FALSE), n, replace = TRUE, prob = c(0.08, 0.92)),
              NA,
              round(runif(n, min = 10, max = 20), 2)
            ),

  moy_S2   = ifelse(
              sample(c(TRUE, FALSE), n, replace = TRUE, prob = c(0.06, 0.94)),
              NA,
              round(runif(n, min = 10, max = 20), 2)
            ),
  # Statut boursier
  boursier = factor(sample(c("Oui", "Non"), n, replace = TRUE)),

  # Région d'origine avec valeurs manquantes - caractère
  region = factor(sample(
    c("Dakar", "Thiès", "Saint-Louis", "Ziguinchor", NA),
    n, replace = TRUE, prob = c(0.35, 0.25, 0.20, 0.15, 0.05)
  )),

  stringsAsFactors = FALSE
)
```

Le typage est défini explicitement pour chaque variable : `factor()` pour les qualitatives, `round(runif())` pour les notes, et `NA` introduits naturellement via `prob` dans `sample()`.

4.2 Question 2 – Création de la variable moyenne_annuelle

Ici, on peut utiliser la fonction `mutate` du package `dplyr`, et `rowMeans` pour le calcul des moyennes en lignes. Le `na.rm = FALSE` ici est volontaire, car ce n'est pas intéréssant d'imputer une moyenne non méritée à un étudiant. On arrondira les resultats à 2 chiffres après la virgule.

```r
 etudiants <- etudiants %>%
mutate(
    moyenne_annuelle = round(rowMeans(
      cbind(moy_S1, moy_S2), na.rm = FALSE), 2)
  )
```

4.3 Question 3 – Création de la variable sur le statut d'admission

On peut utiliser la fonction de base `ifelse` pour créer la variable **Statut**, en prenant la modalité **Non renseigné** pour les moyennes indisponibles.

```r
etudiants$Statut <- ifelse(
  is.na(etudiants$moyenne_annuelle),"Non renseigné",
   ifelse(etudiants$moyenne_annuelle>=12,"Admis","Ajourné")
) %>% factor()
```

4.4 Question 4 – Pertinence de la transformation en facteurs ordonnés

Dans la pratique, il n'y a pas de "hiérarchie" entre les filières. En effet, aucune filière dans l'école n'est au dessus d'une autre, donc il n'est pas pertinent de transformer la variable **`filiere`** en variable catégorielle ordonnée.

##Question 5 – Vérification de la structure complète

Pour vérifier la stucture d'un data frame, on peut utiliser la fonction `str()`:

```r
str(etudiants)
```

```
## 'data.frame':    100 obs. of  9 variables:
##  $ id              : chr  "ETU001" "ETU002" "ETU003" "ETU004" ...
##  $ sexe            : Factor w/ 2 levels "F","M": 2 2 2 1 2 1 1 1 2 2 ...
##  $ filiere         : Factor w/ 3 levels "AS","ISE","ISEP": 3 2 3 2 3 1 1 2 1 3 ...
##  $ moy_S1          : num  14.2 11.2 15.3 12.2 14.9 ...
##  $ moy_S2          : num  18 10.9 18.3 12.8 17.5 ...
##  $ boursier        : Factor w/ 2 levels "Non","Oui": 1 1 2 1 1 2 2 1 2 1 ...
##  $ region          : Factor w/ 4 levels "Dakar","Saint-Louis",..: NA 1 1 3 1 3 2 1 2 4 ...
##  $ moyenne_annuelle: num  16.1 11.1 16.8 12.5 16.2 ...
##  $ Statut          : Factor w/ 3 levels "Admis","Ajourné",..: 1 2 1 1 1 1 1 3 3 1 ...
```

```r
# Types de chaque colonne via dplyr
etudiants %>% summarise(across(everything(), class))
```

```
##          id   sexe filiere  moy_S1  moy_S2 boursier region moyenne_annuelle
## 1 character factor  factor numeric numeric   factor factor          numeric
##   Statut
## 1 factor
```

```r
# Plages de valeurs pour les notes
etudiants %>%
  summarise(
    min_S1 = min(moy_S1, na.rm = TRUE),
    max_S1 = max(moy_S1, na.rm = TRUE),
    min_S2 = min(moy_S2, na.rm = TRUE),
    max_S2 = max(moy_S2, na.rm = TRUE)
  )
```

```
##   min_S1 max_S1 min_S2 max_S2
## 1  10.18  19.97  10.16  19.93
```

Commentaire :

- Les types sont cohérents avec les définitions : `factor` pour les qualitatives et `numeric` pour les notes.
- Les valeurs manquantes sont concentrées sur `region` (manquant non critique) et sur les moyennes.
- Les notes sont dans la plage $[10, 20]$ — aucune anomalie de plage. -Les identifiants sont laissés en type `character`, car chaque identifiant est unique.

4.5 Question 6 – Contrôle automatique

```r
# Valeurs manquantes par colonne
colSums(is.na(etudiants))
```

```
##               id             sexe          filiere           moy_S1
##                0                0                0                8
##           moy_S2         boursier           region moyenne_annuelle
##                5                0                3               12
##           Statut
##                0
```

```r
# Valeurs aberrantes sur les notes
etudiants %>%
  filter(moy_S1 < 0 | moy_S1 > 20 |
         moy_S2 < 0 | moy_S2 > 20)
```

```
## [1] id               sexe             filiere          moy_S1
## [5] moy_S2           boursier         region           moyenne_annuelle
## [9] Statut
## <0 lignes> (ou 'row.names' de longueur nulle)
```

```r
# Types de chaque colonne
sapply(etudiants, class)
```

```
##               id             sexe          filiere           moy_S1
##      "character"         "factor"         "factor"        "numeric"
##           moy_S2         boursier           region moyenne_annuelle
##        "numeric"         "factor"         "factor"        "numeric"
##           Statut
##         "factor"
```

Aucune valeur aberrante n'est détectée sur les notes — elles sont toutes dans la plage $[10, 20]$. Les seules valeurs manquantes concernent `region` et `moyenne_annuelle`, ce qui est cohérent avec nos choix de construction.

4.6 Question 7 – Implications pour les analyses statistiques

4.6.1 Régression linéaire

```r
modele <- lm(moyenne_annuelle ~ filiere + boursier, data = etudiants)
summary(modele)
```

```
##
## Call:
## lm(formula = moyenne_annuelle ~ filiere + boursier, data = etudiants)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -4.8474 -0.8293  0.0668  1.3898  4.1718
##
```

```
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)  15.3145     0.4350  35.208   <2e-16 ***
## filiereISE   -0.2663     0.4979  -0.535    0.594
## filiereISEP   0.3572     0.5469   0.653    0.515
## boursierOui  -0.1071     0.4223  -0.254    0.800
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.966 on 84 degrees of freedom
##   (12 observations effacées parce que manquantes)
## Multiple R-squared:  0.01699,    Adjusted R-squared:  -0.01812
## F-statistic: 0.4839 on 3 and 84 DF,  p-value: 0.6943
```

Pour la regression linéaire, R encode automatiquement les facteurs en variables indicatrices telles que `filiere`, et `boursier`. Les lignes avec `NA` sur `moyenne_annuelle` par exemple sont exclues automatiquement de l'estimation.

4.6.2 Analyse factorielle

```r
# Sélection et nettoyage des variables numériques uniquement
etudiants_num <- etudiants %>%
  select(where(is.numeric)) %>%
  na.omit()

cor(etudiants_num)
```

```
##                      moy_S1     moy_S2 moyenne_annuelle
## moy_S1            1.0000000 -0.1469053        0.6276904
## moy_S2           -0.1469053  1.0000000        0.6778038
## moyenne_annuelle  0.6276904  0.6778038        1.0000000
```

L'ACP (l'Analyse des Composantes Principales) n'accepte que des variables numériques. Les variables `factor` comme `filiere` et `sexe` doivent être encodées en 0/1 au préalable, ou traitées séparément via une **Analyse des Correspondances Multiples** (ACM).